

```
1  #include <stdio.h>
2
3  #define MAX_PROCESSES 5
4  #define MAX_RESOURCES 3
5
6  int is_safe();
7  int available[MAX_RESOURCES] = {3, 3, 2}; // Available instances of each resource
8  int maximum[MAX_PROCESSES][MAX_RESOURCES] = {{7, 5, 3},
9  {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
10 int allocation[MAX_PROCESSES][MAX_RESOURCES] = {{0, 1, 0},
11 {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};
12
13 int request_resources(int process_num, int request[]) {
14     // Check if request can be granted
15     for (int i = 0; i < MAX_RESOURCES; i++) {
16         if (request[i] > available[i] || request[i] > maximum[process_num][i] - allocation[process_num][i])
17             return 0; // Request cannot be granted
18     }
19
20     // Try allocating resources temporarily
21     for (int i = 0; i < MAX_RESOURCES; i++) {
22         available[i] -= request[i];
23         allocation[process_num][i] += request[i];
24         // Update maximum and need matrix if request is granted
25         maximum[process_num][i] -= request[i];
26     }
27
28     // Check if system is in safe state after allocation
29     if (is_safe()) {
30         return 1; // Request is granted
31     } else {
32         // Roll back changes if not safe
33         for (int i = 0; i < MAX_RESOURCES; i++) {
34             available[i] += request[i];
35             allocation[process_num][i] -= request[i];
36             maximum[process_num][i] += request[i];
37         }
38         return 0; // Request is denied
39     }
40 }
41
42 int is_safe() {
43     int work[MAX_RESOURCES];
44     int finish[MAX_PROCESSES] = {0};
```

```

45
46 // Initialize work array
47 for (int i = 0; i < MAX_RESOURCES; i++) {
48     work[i] = available[i];
49 }
50
51 // Check if processes can finish
52 int count = 0;
53 while (count < MAX_PROCESSES) {
54     int found = 0;
55     for (int i = 0; i < MAX_PROCESSES; i++) {
56         if (finish[i] == 0) {
57             int j;
58             for (j = 0; j < MAX_RESOURCES; j++) {
59                 if (maximum[i][j] - allocation[i][j] > work[j])
60                     break;
61             }
62             if (j == MAX_RESOURCES) {
63                 // Process can finish, update work and mark as finished
64                 for (int k = 0; k < MAX_RESOURCES; k++) {
65                     work[k] += allocation[i][k];
66                 }
67                 finish[i] = 1;
68                 found = 1;
69                 count++;
70             }
71         }
72     }
73     if (found == 0) {
74         return 0; // No process can finish, not safe state
75     }
76 }
77
78 return 1; // All processes can finish, safe state
79 }
80
81 int main() {
82     int process_num, request[MAX_RESOURCES];
83     printf("Enter process number (0 to 4): ");
84     scanf("%d", &process_num);
85     printf("Enter resource request (e.g., 0 1 0): ");
86     for (int i = 0; i < MAX_RESOURCES; i++) {
87         scanf("%d", &request[i]);
88     }
89
90     if (request_resources(process_num, request)) {
91         printf("Request granted.\n");

```

